

Plataforma Evalis: Sistema de Gestión de Evaluaciones y Control Docente

Piero Stephano Rosario Sánchez

16 de mayo de 2026

Capítulo 1

Introducción y Objetivos

En el entorno educativo actual, la digitalización de los procesos administrativos y académicos se ha vuelto una necesidad fundamental para optimizar el tiempo del personal docente y directivo. El registro de calificaciones, el control del profesorado y la generación de actas oficiales suelen ser tareas complejas que, si se realizan de forma manual o descentralizada, son propensas a errores y retrasos.

Capítulo 2

1. Base de datos

2.1. Diseño de la base de datos relacional

Para garantizar la integridad, consistencia y escalabilidad de la Plataforma Evalis, se ha diseñado una base de datos relacional robusta utilizando el sistema de gestión de bases de datos MySQL. La estructura de almacenamiento se compone de un conjunto de tablas interconectadas mediante claves primarias y foráneas, permitiendo reflejar fielmente la jerarquía y el flujo operativo de un centro educativo.

A continuación, se describen los módulos y tablas principales que conforman el esquema del sistema:

2.1.1. Gestión de Usuarios y Roles de Personal

El núcleo del control de acceso y de la información biográfica se compone de tablas genéricas especializadas a través de relaciones heredadas o de cardinalidad uno a uno:

persona: Tabla base que centraliza la información común de todos los individuos del centro (DNI, nombre, apellidos, datos de contacto). Evita la duplicidad de datos en el sistema.

estudiants / professor / administradors / directiva: Tablas especializadas que se vinculan directamente con la tabla persona mediante el uso del DNI como clave. Almacenan información exclusiva de cada rol, como el NIA en el caso de los estudiantes o las competencias contractuales en el de los profesores.

usuari / login_logs: Entidades encargadas de gestionar las credenciales de autenticación cifradas, el control de las sesiones activas (sessions) y el registro de auditoría de los accesos al sistema.

2.1.2. Estructura académica y plan de estudios

Para la organización de las clases y los contenidos didácticos que se imparten, se han implementado las siguientes relaciones:

grup_classe: Define los diferentes grupos o aulas del centro (por ejemplo, 1r BATX A).

assignatura / modul / unitat: Representan la jerarquía del plan docente. Un módulo se compone de diversas asignaturas, las cuales a su vez se subdividen en unidades didácticas configurables.

grupclasse_assignatura / professor_grup_classe: Tablas intermedias que resuelven las relaciones de muchos a muchos. Permiten asociar qué asignaturas se imparten en qué grupos y qué profesores tienen asignados dichos grupos.

2.1.3. Evaluación, Asistencia e Incidencias

El registro diario del rendimiento y el comportamiento del alumnado se consolida a través de un bloque de tablas operativas:

notes: Es la tabla central del módulo de evaluación. Almacena las calificaciones asociando el identificador de la unidad didáctica (uf_id), el NIA del estudiante y el valor numérico de la nota.

estado_evaluación: Controla de manera global si el sistema se encuentra en un periodo abierto para la inserción de notas o si está bloqueado.

gestio_absencies / asistencia_per_hora: Registran las faltas de asistencia y los retrasos cometidos por los alumnos en franjas horarias específicas.

observacions_incidencies: Espacio dedicado para que el equipo docente anote amonestaciones o comentarios de seguimiento pedagógico.

2.2. Dificultades Encontradas y Soluciones Aplicadas

El desarrollo del modelo de datos de la Plataforma Evalis supuso un reto organizativo y técnico considerable. En primer lugar, cabe destacar que la planificación inicial del proyecto estaba diseñada para ser ejecutada por equipos de dos integrantes. Al tener que asumir el desarrollo del sistema de manera completamente individual, la carga de trabajo en el diseño y la normalización de la base de datos se duplicó, obligando a priorizar la eficiencia y a resolver de forma autónoma diversos problemas técnicos complejos.

A continuación, se detallan los principales inconvenientes surgidos durante la elaboración de la base de datos y cómo se solventaron:

Problema con el diseño de claves primarias compuestas: Al modelar la tabla `notes`, inicialmente se planteó usar un ID autonumérico único como clave primaria. Sin embargo, esto permitía que el sistema pudiera registrar por error más de una nota para un mismo alumno en la misma unidad formativa.

Solución: Se reestructuró la tabla aplicando una clave primaria compuesta por la combinación de `nia` (alumno) y `uf_id` (unidad). De este modo, el propio motor de MySQL impide la duplicidad de registros a nivel físico y garantiza la consistencia de los datos históricos.

Sincronización de datos heredados (Especialización de roles): El diseño requería que los profesores, estudiantes y administradores compartieran datos comunes (como nombre, apellido o DNI) pero mantuvieran campos exclusivos de su rol. Al principio, la separación de las tablas (`persona`, `estudiantes`, `professor`) provocaba problemas de sincronización si se borraba o modificaba un usuario.

Solución: Se implementó una arquitectura de herencia en la base de datos, donde el DNI actúa como Clave Primaria en `persona` y, a su vez, como Clave Foránea (FK) con restricción de borrado y actualización en cascada (`ON DELETE CASCADE / ON UPDATE CASCADE`) en las tablas hijas. Esto asegura que cualquier cambio en los datos maestros se replique instantáneamente en todo el sistema.

Inconsistencias al recuperar datos debido a valores nulos (NULL): Al realizar las primeras pruebas de carga para el boletín de calificaciones, la consulta fallaba o ignoraba a los alumnos que todavía no tenían ninguna nota registrada en la tabla `notes`, ya que se utilizaba un enlace de tipo `INNER JOIN`.

Solución: Se sustituyó la lógica de las consultas por un `LEFT JOIN` combinado con la función `IsDBNull` en el entorno de desarrollo (VB.NET). Con esto se logró que la aplicación liste siempre a la totalidad de los alumnos matriculados en el grupo, mostrando la calificación en blanco si aún no ha sido evaluado, en lugar de romper el flujo de la pantalla.

Capítulo 3

Desarrollo de la Aplicación Móvil

3.1. Enfoque Centrado en la Experiencia de Usuario (UX)

A diferencia del módulo de escritorio de la Plataforma Evalis (diseñado para la gestión operativa y carga masiva de datos por parte del personal docente), la aplicación móvil se concibió única y exclusivamente como un portal de servicios orientado al estudiante. El objetivo principal de este desarrollo es empoderar al alumno, ofreciéndole una interfaz moderna, intuitiva y accesible que centralice toda su vida académica en la palma de su mano.

Para lograr una experiencia de usuario (UX) óptima y adaptada a los estándares actuales, la aplicación implementa las siguientes características de diseño y accesibilidad:

Personalización de interfaz (Tema Oscuro / Negro): Para mejorar el confort visual en entornos de baja luminosidad y optimizar el consumo de batería en pantallas de tecnología OLED/AMOLED, se desarrolló un sistema de gestión de temas que permite al usuario alternar entre el modo claro tradicional, un tema oscuro y un modo negro puro según sus preferencias.

Navegación Fluida y Adaptable: Se sustituyeron las estructuras rígidas y los grandes contenedores de escritorio por un diseño móvil nativo basado en paneles limpios y menús deslizantes, garantizando que la consulta de datos sea rápida, intuitiva y libre de fricciones.

3.2. Módulos y Funcionalidades de la App Móvil

Al estar la aplicación móvil enfocada por completo en la experiencia del alumno, todas sus funciones se estructuran para facilitarle el acceso a la infor-

mación y agilizar la interacción con el centro. Los módulos principales implementados son los siguientes:

Panel de Calificaciones y Notificaciones en Tiempo Real: El alumno puede consultar sus notas actualizadas al instante. Además, el sistema integra un servicio de alertas que envía una notificación push al dispositivo móvil en el momento exacto en que un profesor publica o modifica una calificación en la base de datos.

Módulo de Profesorado y Búsqueda en Tiempo Real: Para facilitar la localización y el contacto con los docentes, la aplicación incluye una pantalla que lista a los profesores vinculados al alumno, organizándolos de forma clara según la asignatura específica que le imparten. Además, se ha implementado una barra de búsqueda inteligente con filtrado en tiempo real, lo que permite encontrar a cualquier profesor al instante conforme se va tecleando su nombre.

Estadísticas Académicas y Rendimiento: El módulo de analítica calcula y muestra de forma gráfica la nota media del estudiante por asignatura, permitiéndole realizar un seguimiento visual de su progreso y rendimiento a lo largo del curso.

Control de Prácticas en Empresa (FCT): Espacio dedicado al seguimiento del módulo de Formación en Centros de Trabajo, donde el alumno puede monitorizar de forma exacta el cómputo acumulado de sus horas de prácticas realizadas en la empresa colaboradora.

Expediente e Información Académica: Permite el acceso al historial de estudios finalizados de cursos anteriores, así como la consulta de datos esenciales del perfil del alumno y el aula asignada.

Generación y Descarga de Boletín en PDF: Mediante la integración de un motor ligero de renderizado de documentos, el estudiante puede exportar e imprimir su boletín oficial de calificaciones directamente en formato PDF desde el propio terminal, sin perder el formato institucional.

3.3. Dificultades Técnicas en el Desarrollo Individual

Desarrollar una aplicación con tal densidad de funcionalidades de manera completamente individual (asumiendo en solitario el trabajo originalmente planificado para un equipo de dos personas) supuso afrontar retos técnicos complejos, especialmente en la gestión y optimización de los datos:

Optimización y rendimiento en la carga del expediente completo: Al abrir el perfil o el expediente, la aplicación móvil debe realizar múltiples consultas a la base de datos MySQL para traer a la vez las notas actuales, las

medias calculadas, el estado de las FCT y los datos del aula. Hacer esto con peticiones separadas colapsaba la conexión y hacía que la app tardara varios segundos en cargar la pantalla.

Solución: Se diseñaron consultas optimizadas utilizando vistas en la base de datos y uniones avanzadas (LEFT JOIN). De este modo, el servidor procesa toda la información en una sola petición estructurada, consiguiendo que el expediente y las estadísticas del alumno se muestren de forma instantánea al abrir la app.

Sincronización del sistema de alertas sin saturar el servidor: Conseguir que el móvil se entere de una nueva nota sin saturar la base de datos con peticiones constantes (polling) desde el dispositivo del alumno era inviable para el rendimiento del sistema.

Solución: Se diseñó una lógica eficiente basada en eventos. La aplicación móvil comprueba de forma inteligente los cambios de fecha y estado en las tablas de la base de datos para disparar la notificación local únicamente cuando se detecta una inserción real en la tabla notes que afecte al NIA del alumno autenticado.

Desarrollo de la Aplicación de Escritorio

3.4. Enfoque Arquitectónico y Modular

La aplicación de escritorio de la Plataforma Evalis constituye el núcleo administrativo y operativo del sistema. A diferencia de la versión móvil (orientada exclusivamente a la consulta por parte del alumnado), el entorno de escritorio está diseñado específicamente para el personal docente, directivo y de administración. Su propósito principal es ofrecer una herramienta de alto rendimiento que simplifique la carga masiva de datos, el control de las evaluaciones y la gestión global del centro.

Para garantizar la mantenibilidad del código y permitir futuras expansiones sin necesidad de reescribir la aplicación, se ha optado por un desarrollo modular en Visual Basic .NET utilizando Windows Forms. La interfaz se articula mediante controles de usuario personalizados (UserControls) que se cargan de forma dinámica en un contenedor principal, evitando la apertura caótica de múltiples ventanas y unificando la experiencia de usuario.

3.5. Gestión de Roles y Control de Acceso

Para garantizar la seguridad de la información académica y evitar alteraciones no autorizadas en las calificaciones, la plataforma implementa un sistema de control de acceso basado en roles. Dependiendo del tipo de cuenta con la que el usuario inicie sesión en la aplicación, el sistema habilitará o bloqueará dinámicamente las diferentes pestañas y funciones de la interfaz.

A continuación, se describen las responsabilidades, funcionalidades y privilegios asignados a cada perfil operativo de la versión de escritorio:

3.5.1. Rol: Profesor

Es el perfil operativo básico dentro del entorno docente. Sus funciones están estrictamente limitadas a la gestión y evaluación de las asignaturas que tiene asignadas en su horario lectivo:

Volcado de Calificaciones mediante DataGridView: Permite introducir y almacenar las notas de los alumnos matriculados en sus grupos de clase de forma masiva a través de una tabla dinámica interactiva (dgvNotas).

Consulta del Claustro Docente: Dispone de una vista exclusiva para listar y consultar a todos los profesores del centro con sus respectivas especialidades.

Restricción Temporal de Edición: El profesor solo tiene permitido insertar o modificar calificaciones cuando el periodo de evaluación se encuentra en estado "ABIERTO". Si el periodo está cerrado, la interfaz bloquea automáticamente la edición del DataGridView, impidiendo cualquier cambio físico en la tabla notes.

3.5.2. Rol: Jefe de Estudios

Este perfil dispone de permisos administrativos intermedios enfocados en la supervisión académica del centro educativo y la gestión de los plazos temporales:

Control del Periodo Evaluativo: Es el encargado directo de modificar la tabla estado_evaluacion, abriendo o cerrando los plazos de entrega de notas para todo el claustro docente con un solo clic.

Exportación de Actas Oficiales: Cuenta con la capacidad de generar y exportar documentos con las calificaciones consolidadas de todos los alumnos pertenecientes a cualquier clase del centro para su posterior tramitación administrativa.

Privilegios Docentes Expandidos: Reúne las mismas capacidades de inserción y edición de notas que un profesor (a través de la misma funcionalidad de cuadrícula de datos), pero con la ventaja jerárquica de poder editar calificaciones o supervisar actas bloqueadas si la situación lo requiere.

3.5.3. Rol: Administrador (Admin)

Es el rol con mayor jerarquía dentro del ecosistema de la plataforma. Posee control total sobre el software, las estructuras maestras y las herramientas de auditoría de seguridad:

Supervisión Total (Full Access): Hereda y puede ejecutar de forma nativa la totalidad de las funciones de los roles de Profesor y Jefe de Estudios de manera conjunta y sin ningún tipo de restricción.

Interoperabilidad de Datos Docentes (JSON): Dispone de una herramienta exclusiva para realizar extracciones masivas de la tabla professor y exportar la información completa del claustro en formato JSON para integraciones externas.

Auditoría de Seguridad y Accesos (XML): Para garantizar la seguridad del sistema frente a accesos indebidos o registrar acciones críticas, el administrador puede realizar consultas avanzadas sobre la tabla login_logs y exportar el historial completo de conexiones de cualquier persona del centro en un archivo estructurado XML.

3.6. Dificultades Técnicas en el Desarrollo Individual

El principal y más crítico desafío durante el desarrollo de la versión de escritorio de la Plataforma Evalis fue el factor tiempo. Al tener que asumir de forma estrictamente individual un proyecto cuya envergadura, alcance y diseño estaban planificados originalmente para ser ejecutados por un equipo de dos integrantes, el cronograma de trabajo se vio severamente ajustado.

Esta limitación temporal obligó a aplicar una estrategia de desarrollo basada en los siguientes puntos:

Sobrecarga en la fase de desarrollo y maquetación: Compaginar en solitario el diseño de la interfaz gráfica en Windows Forms, la programación de la lógica en VB.NET y la estructuración de los roles de acceso duplicó las horas de dedicación necesarias en comparación con un entorno de trabajo colaborativo.

Solución y Mitigación: Se aplicó una estricta priorización de funcionalidades (enfoque MVP o Producto Mínimo Viable), asegurando que los módulos críticos —como el volcado de notas mediante el DataGridView y el control de accesos por rol— estuvieran completamente operativos y pulidos antes de avanzar con los extras del sistema. Se optimizaron las jornadas de desarrollo mediante una planificación modular para garantizar la entrega puntual del proyecto final dentro del plazo establecido.

Capítulo 4

Arquitectura de Comunicación y Seguridad (API PHP)

La aplicación móvil no realiza conexiones directas a la base de datos centralizada, protegiendo así la integridad del servidor MySQL. En su lugar, se ha diseñado e implementado una arquitectura cliente-servidor basada en una API de servicios web en PHP.

El flujo de comunicación y el intercambio de información sigue un protocolo seguro estructurado en tres niveles:

1. Peticiones por URL: La aplicación móvil solicita información (como el expediente, las notas o el perfil) realizando peticiones HTTP a URLs específicas que apuntan a los archivos PHP alojados en el servidor.
2. Validación de Identidad mediante Tokens: Para evitar el robo de información o accesos no autorizados, cada petición debe incluir un Token de Seguridad único generado tras el inicio de sesión. Los archivos PHP interceptan este token y comprueban su validez antes de procesar cualquier orden. Si el token es inválido o ha expirado, el servidor deniega el acceso automáticamente.
3. Procesamiento y Respuesta: Una vez superada la barrera de seguridad del token, el script PHP interactúa de forma segura con MySQL, extrae los datos solicitados y los envía de vuelta al dispositivo móvil de manera estructurada.

4.1. Dificultades Técnicas en el Desarrollo Individual

Desarrollar un ecosistema con este nivel de robustez y seguridad de manera completamente individual (asumiendo en solitario el trabajo de dos personas) supuso afrontar retos técnicos complejos en la sincronización de las sesiones:

Gestión y persistencia del Token de Seguridad en el dispositivo: Al implementar la seguridad por tokens, un problema recurrente era que el alumno perdía el acceso o tenía que loguearse continuamente cada vez que la app móvil cambiaba de pantalla o se minimizaba, debido a la volatilidad de la memoria del terminal.

Solución: Se programó un sistema de almacenamiento seguro y local en la aplicación móvil que retiene el token encriptado de forma persistente mientras la sesión esté activa. La app adjunta este token de forma automática en las cabeceras de las URLs de consulta hacia los archivos PHP, garantizando una navegación fluida, segura y transparente para el usuario.

Sincronización del sistema de alertas sin saturar el servidor: Conseguir que el móvil se entere de una nueva nota sin saturar la base de datos con peticiones constantes (polling) desde el dispositivo del alumno era inviable para el rendimiento del sistema.

Solución: Se diseñó una lógica eficiente basada en eventos. La aplicación móvil comprueba de forma inteligente los cambios de fecha y estado a través de la API en PHP para disparar la notificación local únicamente cuando se detecta una inserción real en la tabla notes que afecte al NIA del alumno autenticado.

4.2. Estrategia de Mitigación y Priorización del Alcance

Como solución general ante la falta de tiempo derivada de asumir el proyecto de manera estrictamente individual (cuando estaba planificado para dos personas), se adoptó una metodología de desarrollo ágil orientada a la priorización del alcance. En lugar de intentar abarcar todas las funcionalidades de manera superficial, la estrategia se centró en asegurar la máxima robustez y un acabado profesional en el núcleo del ecosistema de la Plataforma Evalis.

Para gestionar de forma eficiente el calendario de entregas, se clasificaron los objetivos del desarrollo bajo los siguientes criterios de prioridad:

Elementos de Prioridad Crítica (Desarrollo Completo): Se priorizó el diseño físico de la base de datos centralizada en MySQL, la arquitectura de seguridad basada en Tokens a través de la API en PHP para la app móvil, y el control estricto de accesos por roles en el entorno de escritorio de

VB.NET. De igual forma, se aseguró el correcto funcionamiento del volcado masivo de calificaciones en el DataGridView y la descarga del boletín en PDF desde el terminal móvil, al ser las herramientas esenciales para el flujo de trabajo del centro educativo.

Elementos de Prioridad Secundaria (Postergados para Líneas Futuras): Aquellas funcionalidades de carácter cosmético, los sistemas de mensajería interna no críticos o las optimizaciones avanzadas de la interfaz visual que requerían una alta inversión de tiempo y no aportaban valor inmediato a la operatividad del sistema, se delegaron deliberadamente para futuras actualizaciones del software.

Esta toma de decisiones estratégica permitió mitigar eficazmente el impacto del tiempo en contra, garantizando que el producto final entregado sea un software real, estable, completamente seguro y funcional en todas sus plataformas, cumpliendo rigurosamente con los estándares de calidad exigidos.